

Air-Gapped / Offline Desktop Video Annotation Application | NREIP 2026

The Problem The lab I interned at had a major, ongoing computer vision project that ran inference on real-time video feed. The models they had trained were effective, but the data labeling process was gradually becoming an insurmountable bottleneck. The training video footage, acquired off-site, could be as short as 15 minutes or multiple hours long. So, software engineers took hours out of the week to manually scrub through long videos, crop events into smaller clips, and mark coordinates frame by frame. Working with sensitive data meant that the entire process needed to be air-gapped (completely isolated from the cloud, managed locally). This made dataset management the most difficult of all. Multiple people had worked on the labeling effort over the years, without any standardized folder structure or labeling format — making it difficult to trace labeled data to its source, discern which data was labeled/unlabeled, and generate new datasets to train more accurate models.

With only 4 weeks left in my internship, I turned a 4-hour weekly labeling assignment into a new project sprint: building an offline application that streamlined the entire video-labeling lifecycle, from high-speed scrubbing and AI-assisted annotation to standardized, YOLO-ready dataset export.

What I Built I started by interviewing each of the engineers who had worked on labeling, to understand the pain points. Then, I built a single locally run desktop application in Python with Qt and OpenCV that owns the whole path. The user scrubs video and marks events. Frame extraction is progressive — dense at the start of an event, sparser afterward — with random sampling of non-event frames. An events gallery shows every event in a video alongside its labeling progress. The labeling workspace supports bounding boxes and keypoints, keypoint propagation across a frame range, and pose-estimation pre-labeling that selects GPU or CPU automatically. Export produces event-grouped output or YOLO format, with event-aware train/validation/test splits that keep frames from the same event on one side. The tool enforces one folder layout and offers an opt-in repair that plans every file move and shows the plan for approval before acting. I wrote roughly 330 tests, and I rendered the interface offscreen and reviewed screenshots each round. I drove development with Claude Code: detailed specs, context carried between sessions, and

My Impact Labeling reached about three times its prior speed, cutting a weekly eight-hour workflow to two across 100GB of video. Every image and label traces back to its source video and exact frame. I documented the tool's remaining limitations for whoever picks it up next. My recommendation for the next stage is a confidence-thresholded review loop, where the model annotates every frame and an analyst reviews only what it flags as uncertain.

Ingestion Pipeline for Unstructured PDFs | NREIP 2026

The Problem

A Navy rapid prototyping lab maintained several hundred reports containing a mix of tables, structured sections, and narrative text. Analysts manually read these reports to produce numerical estimates, but the documents followed inconsistent formats: the available fields varied between reports, and the location of information on each page could not be predicted. Before automated extraction or prediction could be attempted, the documents needed to be converted into a consistent, machine-readable representation.

What I Built

I scoped the larger problem into three stages: ingestion, extraction, and prediction, then defined a three-week sprint around the ingestion stage. I implemented a layout-aware pipeline using Docling to analyze each page, identify headings, paragraphs, and tables, apply OCR when necessary, and convert the reports into structured Markdown while preserving document organization and table information. I also researched and documented an architecture for the remaining extraction and prediction stages.

My Impact

I transformed a loosely defined automation goal into a concrete development roadmap and delivered the first foundational component. Instead of building a model directly on inconsistent PDF files, the pipeline gives downstream systems a standardized representation of every report. I also identified the need for validated extraction, historical outcome data, and domain expertise before prediction modeling, leaving the team with both working ingestion software and a path for continuing the project.

DermLens

Helped develop a computer-vision-enabled teledermatology platform by conducting user research, building a longitudinal dataset of more than 5,000 clinical images, and fine-tuning CNNs to classify treatment-site erythema.

The Problem

DermLens is designed to make follow-up for non-surgical skin cancer treatment more accessible. Patients undergoing topical treatment need their treatment sites monitored over time, traditionally requiring repeated in-person visits. The challenge is determining how photos and patient-reported symptoms collected remotely could provide physicians with useful information about treatment progression without replacing physician oversight.

What I Built

I have contributed across the project from its early user-research phase through computer vision development. I interviewed patients about their treatment experience and technology habits, helped translate the clinical workflow into application features and wireframes, and assisted dermatology clinic staff in assembling a custom longitudinal dataset of more than 5,000 treatment images. On the machine learning side, I fine-tuned and evaluated convolutional neural networks for classification of erythema, or treatment-site redness, as an initial step toward computer-assisted monitoring.

My Impact

My work helped move DermLens from a clinical idea toward a data-driven system that can be evaluated technically. I have helped connect patient and physician needs to the application's design while building the dataset and baseline computer vision capabilities needed for future longitudinal monitoring. The project has also allowed me to apply computer vision to a problem where model performance must ultimately be evaluated in the context of real clinical decisions.

HouseMeds | Household Prescription Medication Cost Optimizer

Performing user research and backend development for a household prescription-cost optimization platform that compares medication prices across pharmacies to identify lower-cost purchasing strategies.

The Problem

Cash prices for the same prescription can vary substantially between pharmacies, and finding the lowest price becomes more complicated when optimizing medications for an entire household rather than for a single prescription. HouseMeds is designed to help families compare their medications together and determine a lower-cost purchasing strategy across pharmacies.

What I Built

I began by interviewing patients and physicians to understand how people currently purchase prescriptions and where the process creates friction. I assembled a five-person team to translate those findings into a full-stack web application. I am developing the backend using TypeScript and PostgreSQL, integrating pharmacy pricing through REST APIs, and working on the optimization logic that compares medication prices at the household level rather than treating every prescription independently.

My Impact

I have taken the project from an initial healthcare problem through user research, workflow definition, team formation, and early implementation. My role has focused on making sure the

technical design reflects how patients actually manage medications while coordinating the work needed to turn those requirements into a functioning product. Because HouseMeds is still under active development, its impact so far is the creation of a user-informed architecture and development team rather than measured patient savings.

Computer Vision Vehicle Damage Assessment Device | SEAP 2025

Helped transform an open-ended AI concept into a real-time corrosion-detection system by building a custom dataset, fine-tuning a YOLO segmentation model, and deploying it on Raspberry Pi/Google Coral Edge TPU.

The Problem

Our intern team received an unusually open-ended challenge: explore how AI could help Marines assess vehicle damage. There was no predefined user workflow, dataset, or technical approach. Any useful solution also needed to account for field constraints such as mobility, cost, durability, and limited computing resources.

What I Built

After helping narrow the problem to vehicle corrosion detection, our team collected, cleaned, and labeled more than 1,000 images of corrosion to create a custom dataset. We fine-tuned a YOLOv11 image-segmentation model to identify rust and tested the model using live camera input before adapting it for edge hardware including Raspberry Pi and Google Coral. Because the four-person intern team was initially researching many of the same problems independently, I also introduced daily standups and shared workspaces and established more regular communication with our supervising engineer.

My Impact

Our team turned a broad directive to "build an AI-enabled vehicle damage tool" into a working computer vision system capable of identifying corrosion in real time. I helped take the project through the complete applied machine learning workflow—from defining a practical problem and building our own dataset to model fine-tuning and constrained edge deployment—while improving collaboration between team members working in an unfamiliar technical domain.

SEAP Internship 2024 Projects

Designed, reverse-engineered, manufactured, and tested physical components for Marine end users using CAD and additive manufacturing, turning loosely defined requirements into deployable solutions.

The Problem

My first summer at the Navy's Rapid Prototyping Lab centered on practical engineering problems brought in by Marines and machinists. Some requirements were loosely defined, while other projects involved military vehicle components that were difficult to source through conventional replacement channels. The challenge was to understand what the end user actually needed and quickly turn that need into a manufacturable physical solution.

What I Built

I designed, manufactured, and tested a physical component from a machinist's high-level requirements, communicating directly with Marine end users as the design progressed and ultimately supporting its deployment on base. I also used Fusion 360 and SOLIDWORKS to reverse-engineer hard-to-source vehicle components and reproduce them using additive manufacturing.

My Impact

These projects gave Marine users practical physical solutions rather than designs that stopped at the prototype stage. By combining direct conversations with end users, CAD modeling, reverse engineering, fabrication, and iterative testing, I learned how to take an incomplete real-world requirement and carry it through to a part that could actually be used. That experience became the foundation for the user-centered engineering approach I carried into my later Navy and AI projects.